Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also compare across forks or learn more about diff comparisons.

base: 693e53f ← compare: da9691a

1 commit | 1 file changed | 1 contributor

Commits on Jul 12, 2026

リザルト画面実装完了

c0a2509236 committed 28 minutes ago

da9691a

Showing 1 changed file with 84 additions and 35 deletions.

Split Unified

119 group06.py

```
2   2   import os
3   3   import pygame
4   4
    5 + 
5   6   # 実行ファイルのディレクトリにカレントディレクトリを変更（素材やスコアファイルの読み込みエラー防止）
6   7   os.chdir(os.path.dirname(os.path.abspath(__file__)))
7   8
24  25  BLUE = (0, 100, 255)
25  26  RED = (255, 50, 50)
26  27  YELLOW = (255, 215, 0)
    28 + ORANGE = (255, 120, 0) #追加
    29 + CYAN = (255, 122, 0) #追加
    30 +
    31 + FONT_FILE = "PixelMplus12-Regular.ttf" #追加G　使用するドットフォントのファイル名
    32 + HIGHSCORE_FILE = "highscore.txt"
    33 +
    34 + def load_highscore():
    35 +     """ゲーム起動時に最高スコアを読み込む関数"""
    36 +     if os.path.exists(HIGHSCORE_FILE):
    37 +         with open(HIGHSCORE_FILE, "r") as f:
    38 +             try:
    39 +                 return int(f.read())
    40 +             except ValueError:
    41 +                 return 0
    42 +     return 0
    43 +
    44 + def check_and_save_highscore(current_score, current_highscore):
```

```
+      """今回のスコアがハイスコアを超えていたらファイルに保存する関数"""
+      if current_score > current_highscore:
+          with open(HIGHSCORE_FILE, "w") as f:
+              f.write(str(current_score))
+          return current_score  # 新しいハイスコアを返す
+      return current_highscore  # 更新されなければ今のハイスコアをそのまま返す
27 51
28 52
29 53  def main():
36 60      # ゲームの状態管理用変数
37 61      # "TITLE": タイトル画面, "PLAY": ゲーム中, "GAMEOVER": ゲームオーバー（リザルト）画面
38 62      game_state = "TITLE"
39 -
   63 +
40 64      # --- [D君・G君合流用変数] スコア管理の土台 ---
41 65      current_score = 0  # 今回のスコア（D君がゲーム中に加算する）
42 66      high_score = 0     # 最高スコア（G君がファイルから読み込む）
43 -
   67 +      high_score =load_highscore() #ゲーム機同時に最高スコアを読み込む
   68 +
44 69      # ［G君の合流ポイント①: ゲーム起動時に最高スコアを読み込む］
45 70      # 例: high_score = ranking.load_highscore()
46 -      # 暫定処理（ファイルがなければ0、あれば数値を読み込む）
47 -      if os.path.exists("highscore.txt"):
48 -          with open("highscore.txt", "r") as f:
49 -              try:
50 -                  high_score = int(f.read())
51 -              except ValueError:
52 -                  high_score = 0
53 71
   72 +      try: #追加G
   73 +          font = pygame.font.Font(FONT_FILE, 48)
   74 +          small_font = pygame.font.Font(FONT_FILE, 36)
   75 +          large_font = pygame.font.Font(FONT_FILE, 64)
   76 +      except Exception:
   77 +          # 万が一フォントファイルがない場合のフォールバック（エラー落ち防止）
   78 +          print(f"Warning: {FONT_FILE} が見つかりません。デフォルトフォントを使用します。")
   79 +          font = pygame.font.SysFont(None, 48)
   80 +          small_font = pygame.font.SysFont(None, 36)
   81 +          large_font = pygame.font.SysFont(None, 64)
   82 +
54 83      # フォントの用意（E君・G君のUI表示用フォントが決まるまでの暫定）
55 -      font = pygame.font.SysFont(None, 48)
56 -      small_font = pygame.font.SysFont(None, 36)
57 -
   84 +      #追加G　タイマー用の変数準備
   85 +      start_ticks = 0
   86 +      time_left = 30
   87 +
58 88      running = True
59 89      while running:
60 90          # FPS（フレームレート）の設定
73 103                      if event.key == pygame.K_SPACE:
74 104                          # タイトル画面でスペースキーを押したらゲーム開始
75 105                          current_score = 0  # スコアをリセット
   106 +                          time_left = 30
   107 +                          start_ticks = pygame.time.get_ticks() #追加G　タイマー開始
76 108                          game_state = "PLAY"
```

```
77  109
78  110                 elif game_state == "PLAY":
82  114
83  115                 elif game_state == "GAMEOVER":
84  116                     if event.key == pygame.K_SPACE:
85      -                       # ゲームオーバー画面でスペースキーを押したらタイトルに戻る
    117 +                       # 追加G　タイムオーバ画面でスペースキーを押したらタイトルに戻る
86  118                         game_state = "TITLE"
87  119
88  120         # ========================================
95  127             #  [B君の合流ポイント②: プレイヤーの移動更新]
96  128             #  [C君의 合流ポイント①: アイテムの落下更新]
97  129             #  [D君の合流ポイント①: 当たり判定の計算と、current_scoreの加算・減算]
    130 +           elapsed_seconds = (pygame.time.get_ticks() - start_ticks) / 1000 #追加G 残り時間の計算
    131 +           time_left = 30 - elapsed_seconds
    132 +
    133 +           #  追加G　タイムオーバー判定
    134 +           if time_left <= 0:
    135 +               time_left = 0
    136 +               high_score = check_and_save_highscore(current_score, high_score) #ハイスコアを判定して保存
    137 +               game_state = "GAMEOVER"
    138 +
98  139
99      -           # MVP確認用の暫定ルール（エンターキーを押したらゲーム終了・リザルトへ移動）
100     -           # ※本番はD君の判定で「タイムアップ」や「ライフ0」になったら切り替える
101 140             keys = pygame.key.get_pressed()
102 141             if keys[pygame.K_RETURN]:
103 142                 current_score = 150  # テスト用にスコアが入ったと仮定
    143 +               high_score = check_and_save_highscore(current_score, high_score)
104 144                 game_state = "GAMEOVER"
105     -
106     -               #  [G君の合流ポイント②: ゲーム終了時にハイスコアを判定して保存]
107     -               # 例: ranking.check_and_save_highscore(current_score)
108     -               if current_score > high_score:
109     -                   high_score = current_score
110     -                   with open("highscore.txt", "w") as f:
111     -                       f.write(str(high_score))
112     -
    145 +
    146 +           # MVP確認用の暫定ルール（エンターキーを押したらゲーム終了・リザルトへ移動）
    147 +           # ※本番はD君の判定で「タイムアップ」や「ライフ0」になったら切り替える
    148 +                #  [G君の合流ポイント②: ゲーム終了時にハイスコアを判定して保存]
    149 +
113 150         elif game_state == "GAMEOVER":
114 151             pass
115 152
120 157
121 158         if game_state == "TITLE":
122 159             # タイトル画面の描画（E君・F君の素材が来るまでの暫定表示）
123     -           title_text = font.render("FALLING CATCH GAME", True, BLUE)
124     -           start_text = font.render("Press SPACE to Start", True, WHITE)
125     -           screen.blit(title_text, (200, 200))
126     -           screen.blit(start_text, (230, 350))
    160 +           #タイトル画面を（ドット風、中央揃え）
    161 +           title_text = large_font.render("FALLING CATCH GAME", True, ORANGE)
    162 +           start_text = font.render("Press SPACE to Start", True, WHITE)
```

```
                +
                +            title_rect=title_text.get_rect(center=(SCREEN_WIDTH//2,200))   #スタートタイトルの位置設定（画面中央ｘ＝400、ｙ＝200）
                +            start_rect=start_text.get_rect(center=(SCREEN_WIDTH//2,400)) #スタート文字の位置設定（画面中央X＝400、 ｙ＝400）
                +
                +           # 画面に描画
                +            screen.blit(title_text, title_rect)
                +            screen.blit(start_text, start_rect)
127             
128             elif game_state == "PLAY":
129                 # ［F君・B君の合流ポイント：プレイヤー（カゴ）の描画］
130                 # ［F君・C君の合流ポイント：アイテム（果物・爆弾）の描画］
131                 # ［E君の合流ポイント①：画面上部への「現在のスコア」や「残り時間」の文字描画］
                +            time_text = font.render(f"TIME: {int(time_left)}",True, WHITE)#追加G　残り時間の表示
                +            screen.blit(time_text, (20,20))
132             
133                 # MVP確認用の暫定表示
134 -               play_text = font.render("Playing... (Press Enter to Lose)", True, WHITE)
135 -               screen.blit(play_text, (150, 250))
                +            play_text = font.render("Playing... [Enter:Finish]", True, WHITE)
                +            play_rect = play_text.get_rect(center=(SCREEN_WIDTH // 2, 280))
                +            screen.blit(play_text, play_rect)
136             
137             elif game_state == "GAMEOVER":
138                 # ［G君の合流ポイント③：リザルト画面の描画（A君の画面を乗っ取る）］
                +            over_text = font.render("TIME OVER",True,RED)
                +            over_rect = over_text.get_rect(center=(SCREEN_WIDTH // 2, 150))
                +            screen.blit(over_text, over_rect)
139                 # G君が作成した文字配置や、F君が作ったリザルト背景などをここで描画する
140             
141 -               over_text = font.render("GAME OVER", True, RED)
142 -               screen.blit(over_text, (300, 150))
143 -
144                 # スコアとハイスコアの表示（G君の担当箇所）
145                 score_text = small_font.render(f"YOUR SCORE: {current_score}", True, WHITE)
146                 highscore_text = small_font.render(f"HI-SCORE: {high_score}", True, YELLOW)
147 -               screen.blit(score_text, (300, 250))
148 -               screen.blit(highscore_text, (300, 300))
                +            score_rect = score_text.get_rect(center=(SCREEN_WIDTH // 2, 270))
                +            highscore_rect = highscore_text.get_rect(center=(SCREEN_WIDTH // 2, 340))
                +            screen.blit(score_text, score_rect)
                +            screen.blit(highscore_text, highscore_rect)
                +
149             
150                 retry_text = small_font.render("Press SPACE to Title", True, WHITE)
151 -               screen.blit(retry_text, (270, 400))
                +            screen.blit(retry_text, (260, 450))
152             
153             # 画面の更新
154             pygame.display.flip()
```